

Neural Networks Essentials

Representation Learning

Denis C. Ilie-Ablachim

April 22, 2026

Outline

Introduction to Representation Learning

Theoretical Foundations

Metric Learning and Siamese Networks

Variational Autoencoders (VAEs)

Modern Contrastive Learning

Self-Supervised Predictive Methods

GANs for Representation Learning

Practical Considerations

Conclusion and References

What is Representation Learning?

Definition

Representation Learning is the subfield of machine learning that studies how to automatically discover the representations (features) needed for a given task from raw data.

- ▶ Raw data (pixels, waveforms, text tokens) is high-dimensional and noisy
- ▶ A **representation** is a transformation of raw data into a form that makes it easier to extract useful information
- ▶ Formally: learn a function $f_\theta : \mathcal{X} \rightarrow \mathcal{Z}$ that maps input space $\mathcal{X} \subseteq \mathbb{R}^D$ to a representation space $\mathcal{Z} \subseteq \mathbb{R}^d$, typically with $d \ll D$

The Central Role of Features

Intuition: Think of data as a raw diamond. A representation is the cut and polish that reveals its structure.

Traditional ML Pipeline:

1. Collect raw data
2. **Hand-engineer features** (SIFT, HOG, TF-IDF, ...)
3. Train a classifier on those features

Problem: requires **domain expertise**, is brittle, and doesn't transfer well across tasks.

Representation Learning Pipeline:

1. Collect raw data
2. **Learn features automatically** with a neural network
3. Train a classifier on the learned features

Benefit: features **adapt** to the data, **generalize** across tasks, and scale with data.

Why Representation Learning Matters

- ▶ **Reduces human effort:** no need for manual feature engineering
- ▶ **Powers transfer learning:** representations learned on one task (e.g., ImageNet classification) can be reused for another (e.g., medical image segmentation)
- ▶ **Enables few-shot and zero-shot learning:** good representations make it possible to classify unseen categories with very few or even no examples
- ▶ **Improves interpretability:** latent spaces can reveal structure (clusters, continua) in data
- ▶ **Unifies modalities:** shared representation spaces allow comparing text, images, and audio (e.g., CLIP)

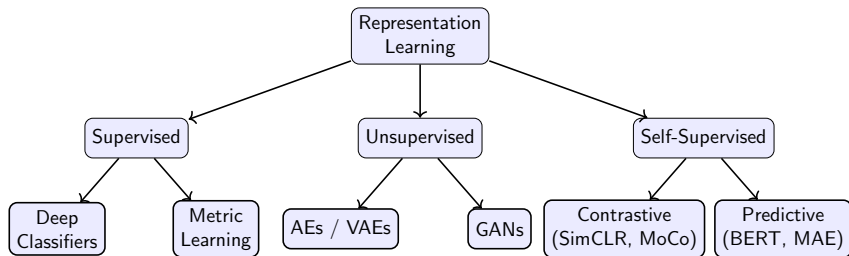
Deep Learning as Hierarchical Representation Learning

- ▶ In a deep neural network, each layer learns a **progressively more abstract** representation of the input
- ▶ Example (ConvNets for images):
 - ▶ Layer 1: edges, textures, color gradients
 - ▶ Layer 2: corners, simple shapes, patterns
 - ▶ Layer 3: object parts (wheels, eyes, wings)
 - ▶ Layer 4+: full objects, scenes, semantic concepts
- ▶ This **compositionality** is a key advantage: complex concepts are built from simpler ones
- ▶ The last hidden layer before the output is often used as the “**learned representation**” or **embedding** of the input

Recap: Autoencoders as Representation Learners

- ▶ In the previous session, we studied **Autoencoders**:
 - ▶ Encoder: $\mathbf{z} = f_{\phi}(\mathbf{x})$ compresses input to latent code
 - ▶ Decoder: $\hat{\mathbf{x}} = g_{\theta}(\mathbf{z})$ reconstructs input from latent code
 - ▶ Training: minimize $\|\mathbf{x} - \hat{\mathbf{x}}\|^2$
- ▶ The bottleneck $\mathbf{z}$ *is* a learned representation — it captures the essential information needed to reconstruct the input
- ▶ **Today's agenda:** we go beyond reconstruction-based methods and explore the broader landscape of representation learning

Taxonomy of Representation Learning Methods



We will explore each branch, with emphasis on **metric learning**, **VAEs**, and **self-supervised** approaches.

The Manifold Hypothesis

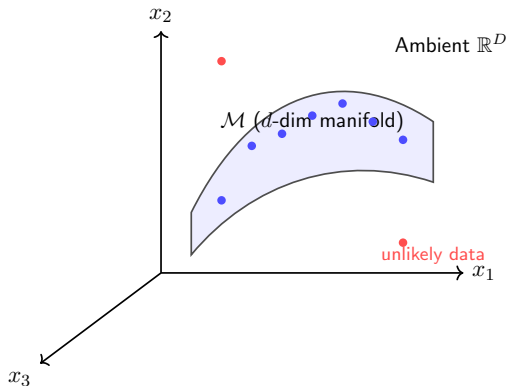
Core Idea

Real-world high-dimensional data (images, speech, text) does not fill the entire ambient space $\mathbb{R}^D$. Instead, it concentrates on or near a **low-dimensional manifold** $\mathcal{M}$ of intrinsic dimension $d \ll D$.

Intuition: An image of a face at 256×256 pixels lives in $\mathbb{R}^{196608}$ (3 color channels). But the set of “natural face images” is parameterized by far fewer variables: pose, expression, lighting, identity — perhaps a few hundred dimensions at most.

Consequence: If we can learn a mapping to coordinates on $\mathcal{M}$, we obtain a compact, powerful representation.

Manifold Hypothesis: Visual Intuition



Data points (blue) cluster near a low-dimensional manifold $\mathcal{M}$. Points far from $\mathcal{M}$ (red) have negligible probability. If data truly lies on a manifold of dimension $d \ll D$, algorithms exploiting that structure are more **sample-efficient** and **robust**.

Linear Representations: PCA

Principal Component Analysis (PCA) is the simplest manifold learning method: it assumes data lies near a **linear subspace**.

Given data matrix $\mathbf{X} \in \mathbb{R}^{N \times D}$ (centered), PCA finds:

$$\mathbf{h} = \mathbf{W}^\top \mathbf{x}, \quad \mathbf{W} \in \mathbb{R}^{D \times d}$$

where columns of $\mathbf{W}$ are the top- d eigenvectors of $\mathbf{X}^\top \mathbf{X}$.

- ▶ **Strength:** closed-form solution, fast, no hyperparameters (aside from d)
- ▶ **Limitation:** cannot capture non-linear structure. A Swiss roll, for instance, cannot be “unrolled” by a linear projection
- ▶ **Connection:** a single-layer autoencoder with linear activations and MSE loss learns the PCA subspace

Non-Linear Representations

To capture curved manifolds, we need **non-linear** mappings:

$$\mathbf{h} = f(\mathbf{x}; \theta)$$

where f is a neural network parameterized by θ .

Examples of non-linear methods:

- ▶ **Kernel PCA**: apply PCA in a kernel-induced feature space
- ▶ **t-SNE / UMAP**: non-parametric methods for visualization (not easily applicable to new data)
- ▶ **Autoencoders**: learn encoder f and decoder g jointly via reconstruction
- ▶ **Deep metric learning**: learn f via pairwise/triplet objectives
- ▶ **Contrastive / self-supervised methods**: learn f by exploiting data augmentations

Neural networks are **universal approximators**: given enough capacity, they can approximate any continuous mapping to arbitrary precision (Cybenko, 1989).

What Makes a Good Representation?

Let $\mathbf{z} = f(\mathbf{x}; \theta)$ be the learned representation. Ideally:

1. **Informativeness:** $\mathbf{z}$ retains enough information about $\mathbf{x}$ to solve downstream tasks. Formally, high mutual information $I(\mathbf{z}; y)$ where y is a task label.
2. **Compactness:** $\dim(\mathbf{z}) \ll \dim(\mathbf{x})$. Compression forces the model to identify essential structure.
3. **Disentanglement:** each coordinate z_i captures a single, independent factor of variation (pose, color, size, ...).
4. **Invariance:** $\mathbf{z}$ is robust to nuisance transformations (lighting, background, noise).
5. **Smoothness:** nearby points in $\mathcal{Z}$ correspond to semantically similar inputs. Small changes in $\mathbf{z}$ produce small, meaningful changes in $\mathbf{x}$.
6. **Transferability:** the representation generalizes to new tasks and domains.

Information-Theoretic Perspective

Representation learning can be framed via the **Information Bottleneck** (Tishby et al., 2000):

$$\min_f I(\mathbf{x}; \mathbf{z}) - \beta I(\mathbf{z}; y)$$

- ▶ $I(\mathbf{x}; \mathbf{z})$: mutual information between input and representation. We want this to be **small** (compression)
- ▶ $I(\mathbf{z}; y)$: mutual information between representation and target. We want this to be **large** (task relevance)
- ▶ β controls the trade-off

Intuition: the ideal representation is a **minimal sufficient statistic** — it throws away as much as possible about $\mathbf{x}$ while keeping everything relevant to y .

This provides a principled view of why bottleneck layers in autoencoders, dropout, and other regularizers help: they limit $I(\mathbf{x}; \mathbf{z})$, forcing the model to retain only essential information.

What is Metric Learning?

Idea

Instead of learning a classifier directly, learn an **embedding function** $f_\theta : \mathcal{X} \rightarrow \mathbb{R}^d$ such that a simple distance metric in embedding space reflects semantic similarity.

Intuition: map data to a space where:

- ▶ similar items are **close** (small $\|f_\theta(x_i) - f_\theta(x_j)\|$)
- ▶ dissimilar items are **far apart** (large $\|f_\theta(x_i) - f_\theta(x_j)\|$)

Why metric learning?

- ▶ Works well when the number of classes is large or not fixed at training time (open-set recognition)
- ▶ Natural fit for **retrieval**, **verification**, and **few-shot learning**
- ▶ The learned embeddings are the **representation**

Siamese Networks: Architecture

- ▶ A **Siamese network** consists of two (or more) identical sub-networks that **share weights**
- ▶ Each sub-network processes one input and produces an embedding
- ▶ A distance function compares the embeddings

Weight sharing is critical: it ensures that the **same transformation** is applied to both inputs, so the distance is meaningful.

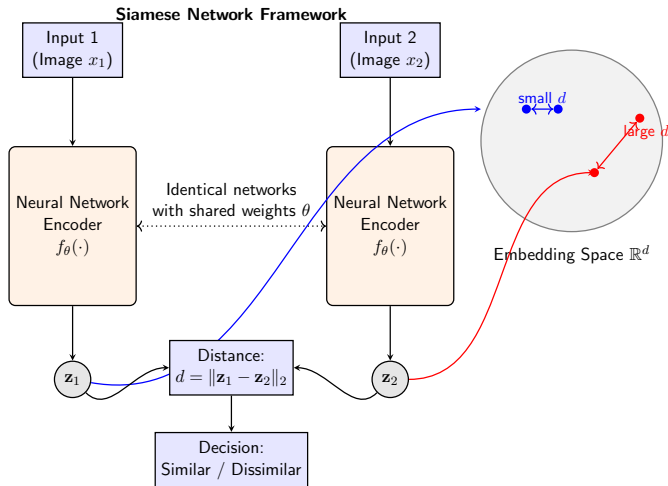
Formally:

Given (x_i, x_j) , compute $\mathbf{z}_i = f_\theta(x_i)$, $\mathbf{z}_j = f_\theta(x_j)$

$$d(x_i, x_j) = \|\mathbf{z}_i - \mathbf{z}_j\|_2$$

Key applications: face verification, signature verification, one-shot learning, visual object tracking.

Siamese Network: Architecture Diagram



Contrastive Loss: Intuition

We are given pairs of examples with a label:

- ▶ $y = 1$: **positive pair** (same class / similar)
- ▶ $y = 0$: **negative pair** (different class / dissimilar)

What we want the loss to do:

- ▶ If $y = 1$ (positive pair): **pull embeddings together** — minimize $d_{ij} = \|\mathbf{z}_i - \mathbf{z}_j\|$
- ▶ If $y = 0$ (negative pair): **push embeddings apart** — but only if they are closer than a **margin** m

Why a margin? Without it, the loss would try to push negative pairs to $d \rightarrow \infty$, which causes:

- ▶ Unbounded embedding norms
- ▶ Wasted capacity: once pairs are well-separated, further separation is useless

The margin m says: *“once negatives are at least m apart, stop pushing.”*

Contrastive Loss: Mathematical Formulation

Let $d_{ij} = \|f_{\theta}(x_i) - f_{\theta}(x_j)\|_2$ and $y \in \{0, 1\}$:

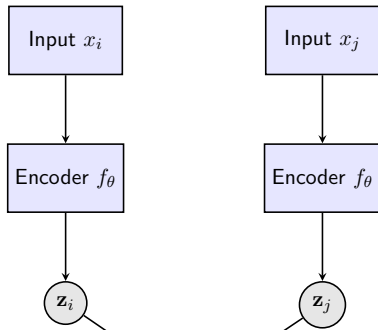
$$\mathcal{L}_{\text{contrastive}} = \frac{1}{2} \sum_{(i,j)} \left[y \cdot d_{ij}^2 + (1 - y) \cdot \max(0, m - d_{ij})^2 \right]$$

Case analysis:

- ▶ **Positive pair** ($y = 1$): $\mathcal{L} = \frac{1}{2}d_{ij}^2$. Gradient pushes $d_{ij} \rightarrow 0$.
- ▶ **Negative pair** ($y = 0$):
 - ▶ If $d_{ij} < m$: $\mathcal{L} = \frac{1}{2}(m - d_{ij})^2 > 0$. Gradient pushes $d_{ij} \rightarrow m$.
 - ▶ If $d_{ij} \geq m$: $\mathcal{L} = 0$. No gradient — pair is already well-separated.

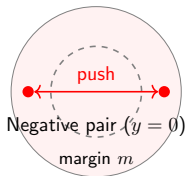
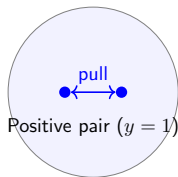
Hyperparameter m is typically set between 0.5 and 2.0, depending on embedding normalization.

Contrastive Loss: Visualization



$$d_{ij} = \|\mathbf{z}_i - \mathbf{z}_j\|_2$$

$$\mathcal{L} = y \cdot d_{ij}^2 + (1-y) \cdot \max(0, m - d_{ij})^2$$



Contrastive Loss: Gradient Analysis

Understanding the gradients helps build intuition:

For a positive pair ($y = 1$):

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}_i} = (\mathbf{z}_i - \mathbf{z}_j)$$

Pushes $\mathbf{z}_i$ toward $\mathbf{z}_j$ with force proportional to distance.

For a negative pair ($y = 0$) when $d_{ij} < m$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}_i} = -(m - d_{ij}) \cdot \frac{\mathbf{z}_i - \mathbf{z}_j}{d_{ij}}$$

Pushes $\mathbf{z}_i$ **away from** $\mathbf{z}_j$. The closer they are (relative to m), the stronger the repulsion.

When $d_{ij} \geq m$: gradient is zero — “mission accomplished.”

Triplet Loss: Motivation

Problem with contrastive loss: it only considers pairs. It doesn't directly enforce the relative ordering:

“anchor should be closer to positive than to negative”

Triplet loss addresses this by considering **three** samples at once:

- ▶ **Anchor** x^a : reference sample
- ▶ **Positive** x^p : same class as anchor
- ▶ **Negative** x^n : different class from anchor

Goal: ensure the anchor is closer to the positive than to the negative by a margin α :

$$d(x^a, x^p) + \alpha < d(x^a, x^n)$$

Triplet Loss: Mathematical Formulation

Define:

$$d_p = \|f_\theta(x^a) - f_\theta(x^p)\|_2^2, \quad d_n = \|f_\theta(x^a) - f_\theta(x^n)\|_2^2$$

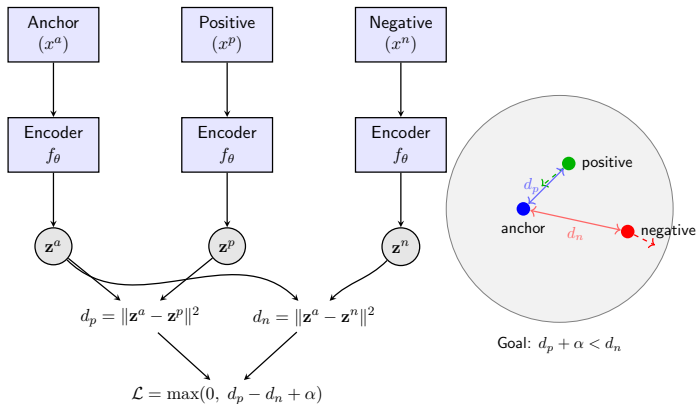
$$\mathcal{L}_{\text{triplet}} = \sum_{(a,p,n)} \max(0, d_p - d_n + \alpha)$$

Case analysis:

- ▶ If $d_n > d_p + \alpha$ (negative is far enough): $\mathcal{L} = 0$. Triplet is already satisfied.
- ▶ If $d_p < d_n < d_p + \alpha$ (negative is close but on the right side): small positive loss pushes d_n further.
- ▶ If $d_n < d_p$ (negative is closer than positive!): large loss — this is a “violated” triplet.

$\alpha > 0$ is a hyperparameter (margin). Typical values: 0.1 to 1.0.

Triplet Loss: Architecture Diagram



Hard Negative Mining

Problem: most triplets in a large dataset are “easy” — the negative is already far from the anchor. These contribute zero loss and zero gradient.

Solution: focus on **hard** or **semi-hard** triplets:

- ▶ **Hard negatives:** $d_n < d_p$ (negative is closer than positive). Strongest learning signal, but can destabilize training.
- ▶ **Semi-hard negatives:** $d_p < d_n < d_p + \alpha$ (negative is farther than positive, but within the margin). The “sweet spot” for stable training.
- ▶ **Easy negatives:** $d_n > d_p + \alpha$. Zero loss, no learning.

Strategies:

- ▶ **Online mining:** within each mini-batch, select the hardest/semi-hard triplets
- ▶ **Offline mining:** periodically compute all embeddings and select hard examples for the next epoch

Contrastive vs. Triplet: Comparison

Aspect	Contrastive Loss	Triplet Loss
Input	Pair (x_i, x_j)	Triplet (x^a, x^p, x^n)
Labels	Binary $y \in \{0, 1\}$	Implicit from triplet
Optimization	Absolute distances	Relative ordering
Margin	On negative distance	On $d_p - d_n$
Sampling	$O(N^2)$ pairs	$O(N^3)$ triplets
Stability	Generally stable	Needs careful mining

In practice:

- ▶ Triplet loss often yields better embeddings because it optimizes *relative* ordering
- ▶ But it requires more careful implementation (mining strategy, batch construction)
- ▶ FaceNet (Schroff et al., 2015) popularized triplet loss for face recognition

Beyond Pairs and Triplets: N-Pair Loss

Limitation of triplet loss: each update only considers *one* negative. Inefficient when many negatives carry useful information.

N-Pair Loss (Sohn, 2016) generalizes to $N - 1$ negatives simultaneously:

$$\mathcal{L}_{\text{N-pair}} = \log \left(1 + \sum_{j \neq i} \exp(\mathbf{z}_i^\top \mathbf{z}_j^- - \mathbf{z}_i^\top \mathbf{z}_i^+) \right)$$

- ▶ Compares the anchor against **all** negatives in the batch simultaneously
- ▶ More informative gradients per update
- ▶ Closely related to the **InfoNCE** loss used in modern contrastive learning (SimCLR, MoCo)
- ▶ Bridges the gap between metric learning and self-supervised learning

Applications of Metric Learning

Face Recognition (FaceNet):

- ▶ 128-dim embeddings for faces
- ▶ Same person $\rightarrow$ small distance
- ▶ Different person $\rightarrow$ large distance
- ▶ Scales to millions of identities

Visual Search / Retrieval:

- ▶ Map products to embeddings
- ▶ “Find similar items” = nearest neighbors in embedding space

Few-Shot Learning:

- ▶ Learn embeddings from base classes
- ▶ Classify new classes using just 1–5 examples via nearest-neighbor

Other applications:

- ▶ Signature verification
- ▶ Speaker identification
- ▶ Sentence similarity (NLP)
- ▶ Drug–target affinity

From Autoencoders to VAEs: Motivation

Recall standard autoencoders:

- ▶ Encoder: $\mathbf{z} = f_{\phi}(\mathbf{x})$ (deterministic mapping)
- ▶ Decoder: $\hat{\mathbf{x}} = g_{\theta}(\mathbf{z})$
- ▶ Loss: $\|\mathbf{x} - \hat{\mathbf{x}}\|^2$

Problems with standard AEs for representation learning:

1. Latent space may have “holes” — regions that don’t map to valid outputs
2. No principled way to **sample** new data points
3. Latent codes can be **arbitrarily scattered** — no structure

Solution: VAE — impose a **probabilistic structure** on the latent space, forcing it to be smooth and well-organized.

VAE: Probabilistic Framework

Generative model:

$$p(\mathbf{x}) = \int p_{\theta}(\mathbf{x}|\mathbf{z}) p(\mathbf{z}) d\mathbf{z}$$

- ▶ Prior: $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$ (standard Gaussian)
- ▶ Likelihood: $p_{\theta}(\mathbf{x}|\mathbf{z})$ parameterized by decoder network

Inference model (encoder):

$$q_{\phi}(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}_{\phi}(\mathbf{x}), \text{diag}(\boldsymbol{\sigma}_{\phi}^2(\mathbf{x})))$$

The encoder outputs **two vectors**: mean $\boldsymbol{\mu}$ and (log-)variance $\log \boldsymbol{\sigma}^2$.

Key distinction from AE: the encoder maps each input not to a single point, but to a **distribution** in latent space.

VAE: The ELBO

We want to maximize $\log p(\mathbf{x})$, but the integral is intractable. Instead, maximize a lower bound:

$$\log p(\mathbf{x}) \geq \underbrace{\mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x}|\mathbf{z})]}_{\text{Reconstruction term}} - \underbrace{D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) \parallel p(\mathbf{z}))}_{\text{Regularization term}}$$

Intuition for each term:

- **Reconstruction:** “the latent code should allow accurate reconstruction.” Equivalent to the standard AE reconstruction loss.
- **KL divergence:** “the learned posterior $q_\phi(\mathbf{z}|\mathbf{x})$ should stay close to the prior $\mathcal{N}(\mathbf{0}, \mathbf{I})$.” This prevents the encoder from collapsing each input to a single distant point — it forces **overlap** and **smoothness** in latent space.

VAE: KL Divergence (Closed Form)

For two Gaussians, the KL has a closed-form expression:

$$D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}) \| p(\mathbf{z})) = -\frac{1}{2} \sum_{j=1}^d \left(1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2 \right)$$

Analyzing each term:

- ▶ μ_j^2 : penalizes means far from zero $\rightarrow$ keeps encodings centered
- ▶ σ_j^2 : penalizes variance far from 1 $\rightarrow$ prevents collapse ($\sigma \rightarrow 0$)
- ▶ $\log \sigma_j^2$: balances the σ_j^2 penalty $\rightarrow$ minimum at $\sigma_j = 1$

In practice, the total VAE loss is:

$\mathcal{L}_{\text{VAE}} = -\mathbb{E}_{q_{\phi}}[\log p_{\theta}(\mathbf{x}|\mathbf{z})] + D_{\text{KL}}(q_{\phi}||p)$, i.e., reconstruction loss + KL penalty.

The Reparameterization Trick

Problem: to train with backprop, we need gradients through the sampling operation $\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})$. But sampling is **not differentiable**.

Solution — reparameterize:

$$\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \quad \mathbf{z} = \boldsymbol{\mu}_\phi(\mathbf{x}) + \boldsymbol{\sigma}_\phi(\mathbf{x}) \odot \epsilon$$

Now $\mathbf{z}$ is a **deterministic function** of $\boldsymbol{\mu}$, $\boldsymbol{\sigma}$, and noise ϵ . Since ϵ doesn't depend on ϕ , we can backpropagate through $\mathbf{z}$ to update ϕ .

Intuition: instead of “sample from a learned distribution” (non-differentiable), rewrite it as “take a fixed noise sample and shift/scale it” (differentiable).

VAE: Latent Space Properties

The KL regularizer gives VAEs a **structured latent space**:

- ▶ **Continuity:** nearby points in latent space decode to similar outputs. You can **interpolate** between any two data points by interpolating their latent codes.
- ▶ **Completeness:** every point sampled from $\mathcal{N}(\mathbf{0}, \mathbf{I})$ decodes to a plausible output. No “dead zones.”
- ▶ **Smoothness:** small perturbations in $\mathbf{z}$ lead to smooth semantic changes in $\mathbf{x}$.
- ▶ **Generation:** sample $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, decode $\hat{\mathbf{x}} = g_{\theta}(\mathbf{z})$ to generate new samples.

These properties make VAE representations excellent for **generative tasks**, **data augmentation**, and **anomaly detection** (high reconstruction error for anomalies).

β -VAE and Disentanglement

β -**VAE** (Higgins et al., 2017) modifies the ELBO by scaling the KL term:

$$\mathcal{L}_{\beta\text{-VAE}} = \mathbb{E}_{q_\phi}[\log p_\theta(\mathbf{x}|\mathbf{z})] - \beta \cdot D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x})||p(\mathbf{z}))$$

- ▶ $\beta = 1$: standard VAE
- ▶ $\beta > 1$: stronger pressure toward the prior $\rightarrow$ encourages **disentanglement**

Disentanglement: each latent dimension z_j controls **one** independent factor of variation (e.g., rotation, color, size, identity).

Why does $\beta > 1$ help? The stronger KL pressure forces the model to use each latent dimension efficiently. Instead of encoding everything into a few dimensions (entangled), it spreads information across dimensions, each capturing a single factor.

Trade-off: higher β may sacrifice reconstruction quality.

Self-Supervised Contrastive Learning: The Big Picture

Key insight: we can learn powerful representations **without any labels** by defining pretext tasks from the data itself.

Contrastive self-supervised learning:

1. Take an input x and create two augmented views: $\tilde{x}_i$ and $\tilde{x}_j$
2. Encode both: $\mathbf{h}_i = f_\theta(\tilde{x}_i)$, $\mathbf{h}_j = f_\theta(\tilde{x}_j)$
3. Train: the pair $(\mathbf{h}_i, \mathbf{h}_j)$ from the **same** image is a **positive pair**; pairs from **different** images are **negative pairs**
4. The model must learn what is **invariant** across augmentations (semantic content) and what **changes** (nuisance factors like crop, color, rotation)

This is a natural evolution from Siamese/triplet ideas to a large-scale self-supervised setting.

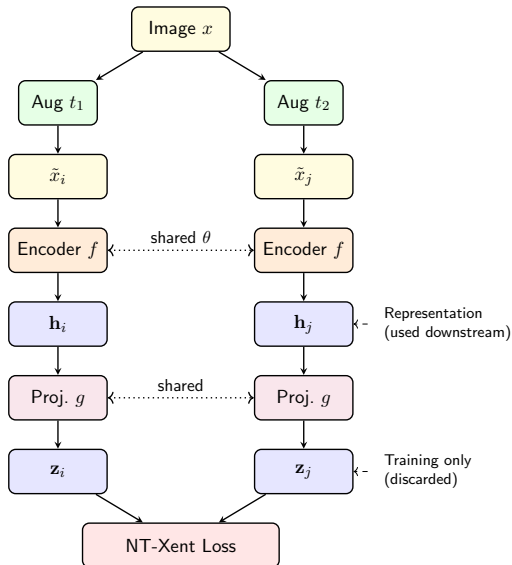
SimCLR: A Simple Framework

SimCLR (Chen et al., 2020) is one of the cleanest contrastive learning frameworks:

1. **Augmentation:** given image x , apply two random augmentations to get $\tilde{x}_i, \tilde{x}_j$ (random crop, color jitter, Gaussian blur, horizontal flip)
2. **Encoder:** apply a backbone network f (e.g., ResNet) to get representations $\mathbf{h}_i = f(\tilde{x}_i)$
3. **Projection head:** apply a small MLP g to get $\mathbf{z}_i = g(\mathbf{h}_i)$ (used only during training)
4. **Contrastive loss (NT-Xent):** for a batch of N images (yielding $2N$ augmented views), maximize agreement between positive pairs relative to all negatives

After training: discard g , use $\mathbf{h} = f(x)$ as the representation.

SimCLR Pipeline



InfoNCE / NT-Xent Loss

For a batch of N images, we get $2N$ augmented views. For the positive pair (i, j) :

$$\ell(i, j) = -\log \frac{\exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_j)/\tau)}{\sum_{\substack{k=1 \\ k \neq i}}^{2N} \exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_k)/\tau)}$$

where $\text{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u}^\top \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}$ (cosine similarity) and τ is a temperature.

Intuition: this is a **softmax classification** problem. Given $\mathbf{z}_i$, classify which of the other $2N - 1$ views is its positive partner. The denominator sums over all negatives, and the loss is lower when $\mathbf{z}_j$ is much more similar to $\mathbf{z}_i$ than any negative.

The temperature τ controls sharpness. Small $\tau \rightarrow$ hard classification (focus on hardest negatives). Typical $\tau \in [0.05, 0.5]$.

Role of Temperature τ

$$\ell(i, j) = -\log \frac{\exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_j)/\tau)}{\sum_{k \neq i} \exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_k)/\tau)}$$

- ▶ $\tau \rightarrow 0$ (**low temperature**): the softmax becomes very peaked. Only the **hardest negative** (most similar to $\mathbf{z}_i$) matters. Risk: training instability.
- ▶ $\tau \rightarrow \infty$ (**high temperature**): all negatives contribute equally. Risk: the model doesn't learn fine-grained distinctions.
- ▶ **Optimal** τ : balances between focusing on hard negatives and maintaining stable gradients. Empirically, $\tau = 0.07$ to 0.5 works well.

Analogy: τ is like the “zoom level” on a distance-based decision boundary. Low τ zooms in (fine-grained), high τ zooms out (coarse-grained).

MoCo: Momentum Contrast

Challenge with SimCLR: needs very large batch sizes (4096+) to get enough negatives.

MoCo (He et al., 2020) decouples batch size from number of negatives:

- ▶ Maintain a **queue** of recent embeddings as a large pool of negatives
- ▶ Use a **momentum encoder** f_ξ (updated as exponential moving average of f_θ):

$$\xi \leftarrow m \cdot \xi + (1 - m) \cdot \theta, \quad m \in [0.99, 0.999]$$

- ▶ Query encoder f_θ : updated by backprop as usual
- ▶ Key encoder f_ξ : updated slowly via momentum $\rightarrow$ consistent representations for the queue

Result: can use 65,536 negatives with a batch size of only 256!

BYOL and Non-Contrastive Methods

Surprise: it turns out negatives may not be strictly necessary.

BYOL (Grill et al., 2020) — Bootstrap Your Own Latent:

- ▶ Two networks: online (encoder + predictor) and target (momentum-updated encoder)
- ▶ Online network predicts the target network's representation
- ▶ **No negative pairs** — yet it doesn't collapse to trivial solutions

Why doesn't it collapse? The asymmetry (predictor + stop-gradient on target + momentum update) creates an implicit regularization that prevents the representation from collapsing to a constant.

Other non-contrastive methods: SimSiam (Chen & He, 2021), Barlow Twins (Zbontar et al., 2021), VICReg (Bardes et al., 2022).

The Importance of Data Augmentations

In self-supervised contrastive learning, the choice of augmentations **defines the learning signal**:

- ▶ Augmentations tell the model: “these variations **don’t matter**” (invariance)
- ▶ The representation learns to encode what is **preserved** across augmentations

Common augmentations (vision):

- ▶ Random resized crop (most important — forces encoding of content, not position)
- ▶ Color jitter / grayscale (invariance to color distribution)
- ▶ Gaussian blur
- ▶ Horizontal flip

Warning: if augmentations are too weak, the model can “cheat” by encoding low-level features. If too strong, the positive pair may no longer share semantic content.

Self-Supervised Learning: Predictive Approaches

Beyond contrastive methods, another powerful paradigm is **masked prediction**:

Core idea: hide part of the input and train the model to predict the missing parts. To do this well, the model must understand the **structure** of the data.

- ▶ **Masked Language Modeling (BERT):** mask random tokens in text, predict them
- ▶ **Masked Image Modeling (MAE):** mask random patches in images, reconstruct them
- ▶ **Next Token Prediction (GPT):** predict the next token given previous ones

These methods learn representations as a “byproduct” of solving the pretext task. The internal representations capture rich semantic information needed for prediction.

BERT: Masked Language Modeling

BERT (Devlin et al., 2019):

1. Take a sentence: ‘‘The cat sat on the [MASK]’’
2. Feed through a Transformer encoder
3. Predict the masked token: ‘‘mat’’ (or ‘‘floor’’, etc.)

Why this works for representation learning:

- ▶ To predict a masked word, the model must understand:
 - ▶ Syntax (grammatical structure)
 - ▶ Semantics (meaning of surrounding words)
 - ▶ World knowledge (what cats do)
- ▶ The internal representations (hidden states) capture all of this
- ▶ Fine-tuning on downstream tasks (classification, QA, NER) achieves state-of-the-art results

15% of tokens are masked. Of those: 80% replaced with [MASK], 10% random token, 10% kept unchanged.

MAE: Masked Autoencoders for Vision

MAE (He et al., 2022): apply the masked prediction idea to images.

1. Divide image into patches (e.g., 16×16)
2. **Mask 75%** of patches (much higher than BERT's 15%!)
3. Encode only the **visible** patches with a ViT encoder
4. Decode all patches (visible + masked) with a lightweight decoder
5. Loss: MSE on the **masked patches only**

Why 75% masking?

- ▶ Images have more spatial redundancy than text
- ▶ With only 25% masking, the model can interpolate pixels without understanding semantics
- ▶ 75% forces the model to develop **holistic understanding** of objects and scenes

Bonus: encoding only 25% of patches makes training 3× faster!

Contrastive vs. Predictive: Comparison

Aspect	Contrastive	Predictive (Masked)
Signal	Augmented pairs	Corrupted input
Architecture	Siamese encoders	Encoder-decoder
Negatives	Required (or tricks)	Not needed
Batch size	Large helps	Standard
Key design	Augmentation strategy	Masking strategy
Risk	Representation collapse	Shortcut learning
NLP	–	BERT, GPT
Vision	SimCLR, MoCo, DINO	MAE, BEiT

Both paradigms produce strong representations. In practice, they are often **complementary**: DINO (contrastive on CLS token) + iBOT (masked prediction on patch tokens) = DINOv2.

GANs: Quick Recap

Generative Adversarial Network (Goodfellow et al., 2014):

$$\min_G \max_D \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))]$$

- ▶ **Generator G :** maps random noise $\mathbf{z} \sim p(\mathbf{z})$ to data space. Goal: fool the discriminator.
- ▶ **Discriminator D :** classifies inputs as real or generated. Goal: detect fakes.
- ▶ At equilibrium (in theory): G produces samples indistinguishable from real data.

For representation learning: both G and D learn useful representations internally.

GANs as Representation Learners

Two ways GANs enable representation learning:

1. Discriminator features:

- ▶ The discriminator must learn to detect subtle differences between real and generated images
- ▶ Its intermediate layers learn features like textures, shapes, object parts
- ▶ These features transfer well to downstream tasks (classification, detection)

2. Generator latent space:

- ▶ The noise vector $\mathbf{z}$ parameterizes the generated output
- ▶ Directions in $\mathbf{z}$ -space often correspond to **semantic attributes** (e.g., adding glasses, changing age, rotating objects)
- ▶ This structure emerges without supervision!

BiGAN / ALI: add an encoder $E : \mathbf{x} \rightarrow \mathbf{z}$ alongside the generator, enabling direct inference of representations from data.

Training Considerations

- ▶ **Embedding normalization:** projecting embeddings onto the unit hypersphere (ℓ_2 normalization) stabilizes contrastive and metric learning. Cosine similarity becomes the natural metric.
- ▶ **Batch construction:** for metric learning, ensure each batch contains multiple examples per class (e.g., P classes $\times K$ examples = PK batch size) to enable meaningful pair/triplet mining.
- ▶ **Learning rate and warmup:** representation learning methods (especially contrastive) benefit from warmup schedules and cosine annealing.
- ▶ **Embedding dimension:** typically 64–2048. Higher dimensions have more capacity but are slower for retrieval. Common: 128 or 256.
- ▶ **Projection head:** in contrastive learning, training the loss on a projection of the representation (not the representation itself) consistently improves downstream performance.

Evaluating Learned Representations

How do we know if a representation is good?

1. Linear probing:

- ▶ Freeze the encoder, train a **linear classifier** on top
- ▶ High accuracy $\Rightarrow$ the representation is linearly separable $\Rightarrow$ strong features
- ▶ This is the standard benchmark in self-supervised learning

2. Fine-tuning:

- ▶ Unfreeze the encoder and train end-to-end on a downstream task
- ▶ Measures the **quality of initialization** provided by the representation

3. Retrieval and ranking:

- ▶ For metric learning: compute Recall@K, mAP, or clustering quality (NMI, ARI)
- ▶ Good embeddings should cluster by semantic class

Visualization of Embeddings

Qualitative evaluation via dimensionality reduction for visualization:

- ▶ **t-SNE** (van der Maaten & Hinton, 2008):
 - ▶ Preserves local neighborhood structure
 - ▶ Creates visually appealing clusters
 - ▶ Perplexity hyperparameter controls cluster size
 - ▶ Caveat: distances between clusters are not meaningful
- ▶ **UMAP** (McInnes et al., 2018):
 - ▶ Preserves both local and some global structure
 - ▶ Faster than t-SNE, scales better
 - ▶ Inter-cluster distances are more meaningful
- ▶ **PCA**: fast, deterministic, but linear — misses non-linear structure

Tip: always combine visualization with quantitative metrics. Pretty t-SNE plots can be misleading!

Common Pitfalls

1. **Representation collapse:** all inputs map to the same point. Happens when contrastive methods lack sufficient negatives or regularization.
2. **Dimensional collapse:** embeddings only use a few dimensions of the latent space. Monitor the effective rank of embedding matrices.
3. **Shortcut learning:** the model exploits low-level cues (e.g., chromatic aberration) instead of semantic content. Mitigate with diverse augmentations.
4. **Overfitting to pretext task:** the representation becomes too specialized for the training objective and doesn't transfer. Use early stopping and validation on downstream tasks.
5. **Wrong similarity metric:** for normalized embeddings, use cosine similarity. For unnormalized, use Euclidean distance. Mixing these leads to poor results.

Transfer Learning with Representations

The ultimate test of a representation: does it transfer to new tasks and domains?

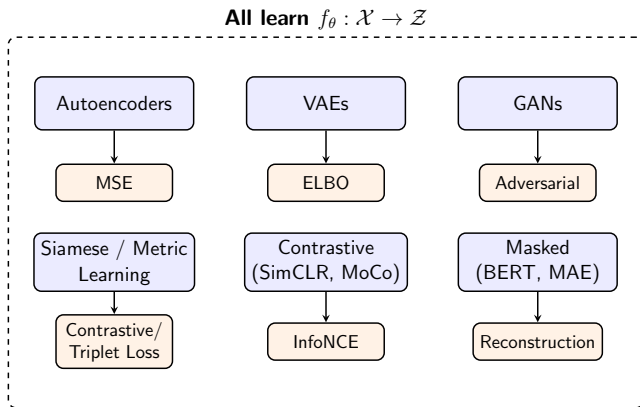
Transfer learning recipe:

1. Pre-train encoder f_θ on large unlabeled dataset (self-supervised)
2. For each downstream task:
 - ▶ **Linear probe:** train only a linear head (fast, tests representation quality)
 - ▶ **Fine-tune:** update all parameters with small learning rate (best performance)
 - ▶ **Few-shot:** use nearest neighbors in embedding space (no training needed)

Foundation models (CLIP, DINOv2, etc.) are the culmination of this approach: pre-train once on massive data, apply everywhere.

Key finding: representations from self-supervised pre-training often rival or exceed supervised pre-training on transfer tasks.

Summary: The Representation Learning Landscape



Key Takeaways

1. **Representation Learning** is the foundation of modern deep learning — every successful model learns useful internal representations.
2. The **manifold hypothesis** justifies why low-dimensional representations work: real data has intrinsically low dimension.
3. **Metric learning** (contrastive/triplet loss, Siamese networks) learns embeddings where distance $\approx$ semantic dissimilarity. Critical for retrieval, verification, and few-shot learning.
4. **VAEs** provide structured, smooth latent spaces with generative capabilities. The ELBO elegantly balances reconstruction and regularization.
5. **Self-supervised methods** (SimCLR, MoCo, MAE, BERT) have revolutionized the field by learning powerful representations **without labels**.
6. A good representation is compact, disentangled, transferable, and robust.

References I

- ▶ **Bengio, Y., Courville, A., & Vincent, P.** (2013). Representation Learning: A Review and New Perspectives. *IEEE TPAMI*.
- ▶ **Tishby, N., Pereira, F., & Bialek, W.** (2000). The Information Bottleneck Method. *Allerton Conference*.
- ▶ **Hadsell, R., Chopra, S., & LeCun, Y.** (2006). Dimensionality Reduction by Learning an Invariant Mapping. *CVPR*.
- ▶ **Schroff, F., Kalenichenko, D., & Philbin, J.** (2015). FaceNet: A Unified Embedding for Face Recognition and Clustering. *CVPR*.
- ▶ **Sohn, K.** (2016). Improved Deep Metric Learning with Multi-class N-pair Loss. *NeurIPS*.
- ▶ **Kingma, D. P. & Welling, M.** (2014). Auto-Encoding Variational Bayes. *ICLR*.
- ▶ **Higgins, I. et al.** (2017). β -VAE: Learning Basic Visual Concepts with a Constrained Variational Framework. *ICLR*.

References II

- ▶ **Chen, T. et al.** (2020). A Simple Framework for Contrastive Learning of Visual Representations (SimCLR). *ICML*.
- ▶ **He, K. et al.** (2020). Momentum Contrast for Unsupervised Visual Representation Learning (MoCo). *CVPR*.
- ▶ **Grill, J.-B. et al.** (2020). Bootstrap Your Own Latent (BYOL). *NeurIPS*.
- ▶ **Devlin, J. et al.** (2019). BERT: Pre-training of Deep Bidirectional Transformers. *NAACL*.
- ▶ **He, K. et al.** (2022). Masked Autoencoders Are Scalable Vision Learners. *CVPR*.
- ▶ **Goodfellow, I. J. et al.** (2014). Generative Adversarial Networks. *NeurIPS*.
- ▶ **Oquab, M. et al.** (2024). DINOv2: Learning Robust Visual Features without Supervision. *TMLR*.
- ▶ **Radford, A. et al.** (2021). Learning Transferable Visual Models From Natural Language Supervision (CLIP). *ICML*.

Thank You!

Questions?

Denis C. Ilie-Ablachim